Assignment D5-V2: Agricultural Robotics – Irrigation Planning

Sisani Francesco

June 16, 2026

Abstract

This report proposes different automated planning solutions using PDDL (Planning Domain Definition Language) for an advanced agricultural irrigation scenario. In this domain, a mobile robot monitors soil moisture levels across various field crops and applies targeted watering actions to maintain crop viability above critical quality thresholds. We explore discrete time scenarios with unlimited and limited water supplies, followed by a continuous-time formulation in PDDL+ to model physical processes such as passive moisture evaporation and sudden drought hazards.

Contents

1 Introduction

This workspace aims to propose different solutions, written in PDDL, for an irrigation scenario in which a robot must monitor the soil moisture of a field (divided into individual crops) and water the singular crops to maintain them above a certain quality threshold. To achieve this, the robot is provided with a limited water supply, the quantity of which depends on the examined subscenario. The robot must also take into account environmental factors, such as passive moisture evaporation due to ambient heat and consequential drought phenomena if one or more crops are left unwatered for too long.

This problem is adapted across multiple distinct configurations and addressed in the domain-problem files contained in their specific directories:

- **Basic_PDDL**: Scenario modeled in discrete time, used to define the basics of the problem. It models the crop-divided field and the robot, the latter being complete of movement capabilities and water supply. Water quantity changes depending on what one of the two problems is being examined.
- **PDDL+**: A scenario modeled in continuous time, designed to approximate a highly realistic physical situation where dynamic environmental factors interact continuously. The priority logic is modified to avoid computationally heavy lookaheads in continuous time while maintaining the same underlying intent. Processes are introduced to represent temporal duration (e.g., passive evaporation and the continuous irrigation act), while events are used to introduce sudden state changes (e.g., a crop instantly entering a drought state).

2 Basic PDDL – Domain File

This scenario utilizes basic PDDL functionalities which are useful both to propose a fundamental solution to the irrigation problem and to lay the groundwork for more complex scenarios. In the code files, the basic mechanics to initialize the environment and grant motion are defined in accordance with standard modeling guidelines.

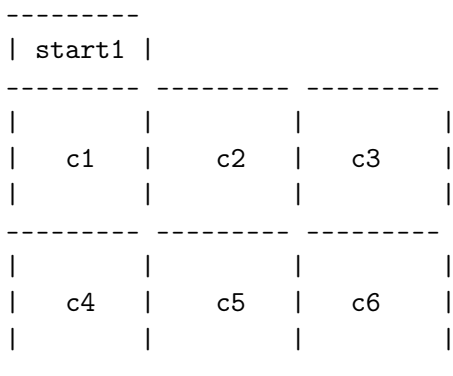
The configuration is split into a pair of text files:

- **Domain file**: Used to initialize types, predicates, functions, and actions that dictate permitted operations within the environment.
- **Problem file**: Used to specify the initial state of the environment, individual object properties, and the target goal state.

In this problem, the domain file uses types, predicates, functions, and actions to model the field layout. The defined types are `robot` and `location`, with the latter split into `crop` and `depot`. The `depot` represents the point where the robot begins its operations, introducing a realistic layout setting.

The predicates `(connected ?l1 ?l2 - location)` and `(at ?r - robot ?l - location)` are used to define neighboring spatial locations and the robot's current position, forming a basic localization environment.

The `connected` predicate encodes connections between sublocations. By mapping them together in the problem file, a precise, confined grid layout is established. The exact topological grid layout utilized across all scenarios is depicted below:



An additional specialized predicate, `(is-priority ?c - crop)`, is utilized to enforce a priority selection strategy. This logic allows the robot to prioritize which crop to irrigate first based on how severe its moisture deficit is. This selection is directly tied to the fluent function `(moisture_level ?c - crop)`, which associates a numerical value with the current soil moisture of each crop. This allows moisture levels to be handled explicitly and discretely via numerical associations.

The actions defined in the domain implement the robot's logic. The primary navigation action is modeled as follows:

```
(:action move
    :parameters (?r - robot ?from - location ?to - location)
    :precondition (and (at ?r ?from) (connected ?from ?to) (not (
   needs_check)))
    :effect (and (at ?r ?to) (not (at ?r ?from)))
)
```

Listing 1: PDDL Navigation Action

In discrete PDDL, `move` is instantaneous and represents a single adjacency hop. The `connected` predicate encodes the field topology, so multi-hop paths are handled natively by chaining multiple sequential `move` steps.

The priority selection mechanism is evaluated via the `check_levels` action:

```
(:action check_levels
    :parameters (?c - crop ?r - robot)
    :precondition (and
        (> (water_supply ?r) 0)
        (forall (?other - crop) (>= (moisture_level ?other) (
   moisture_level ?c)))
    )
    :effect (and
        (is-priority ?c)
        (not (needs_check))
        (forall (?other - crop)
            (when (not (= ?other ?c)) (not (is-priority ?other)))
        )
    )
)
```

Listing 2: PDDL Priority Check Action

The `check_levels` action assigns the `is-priority` flag to whichever crop holds the globally lowest moisture level. The universal quantifier in both the precondition and the effect guarantees

that at most one crop is flagged as a priority at any single time step, effectively enforcing a greedy, moisture-critical-first strategy.

Once a priority crop is identified and targeted, the robot irrigates it through repeated applications of the discrete `irrigate` action:

```
(:action irrigate
    :parameters (?r - robot ?c - crop)
    :precondition (and (at ?r ?c) (is-priority ?c) (> (water_supply ?r) 0) (< (moisture_level ?c) 50))
    :effect (and (increase (moisture_level ?c) 10) (decrease (water_supply ?r) 10) (needs_check))
)
```

Listing 3: PDDL Irrigation Action

`irrigate` applies a discrete 10-unit water increment to the targeted crop while consuming an identical amount from the robot's local tank. The action can only fire if the crop is strictly below the 50-unit threshold and the robot holds sufficient water, modeling irrigation as an iterative process. A fourth action, `sacrifice`, handles the water-exhausted fallback scenario and is discussed in detail below.

3 Basic PDDL – Abundant vs. Limited Scenarios

Both discrete configurations share the same underlying domain logic and an identical six-crop field layout with matching initial moisture characteristics:

Crop	Initial Moisture Level
c1	17
c2	63
c3	42
c4	88
c5	5
c6	71

Three crops (`c5`, `c1`, and `c3`) start below the 50-unit acceptable safety threshold and require immediate water intervention. The absolute minimum water expenditure required to bring all three up to the baseline threshold in the discrete world is:

$$\Delta W = (50 - 5) + (50 - 17) + (50 - 42) = 45 + 33 + 8 \rightarrow 86 \text{ units} \tag{1}$$

In the abundant water scenario (`Irrigation_problem.pddl`), the robot is provisioned with an ample water supply of 500 units, far exceeding the mandatory minimum. The planner ignores the `sacrifice` action entirely and produces a zero-sacrifice solution, following the priority sequence $c5 \rightarrow c1 \rightarrow c3$ dictated by `check_levels`. The objective metric `(minimize (num_sacrificed))` evaluates precisely to 0, and water capacity limits do not restrict plan selection.

In the limited scenario (`Irrigation_limited_problem.pddl`), the available water capacity is reduced to 80 units – 6 units fewer than the minimum physically required to save all crops. Consequently, the fourth fallback action, `sacrifice`, becomes active:

```
(:action sacrifice
    :parameters (?c - crop ?r - robot)
```

```
    :precondition (and (at ?r ?c) (= (water_supply ?r) 0) (not (sacrificed ?c)))
    :effect (and (sacrificed ?c) (increase (num_sacrificed) 1))
)
```

Listing 4: PDDL Sacrifice Action

The `sacrifice` action permanently marks a crop as lost once the robot's water supply is exhausted, allowing the global goal state formula `(forall (?c - crop) (or (>= (moisture_level ?c) 50) (sacrificed ?c)))` to evaluate to true. Combined with the minimization metric, this forces the planner to search for a resource allocation that minimizes total losses.

The resulting sequential plans generated by the solver for both subscenarios are contrasted below.

3.1 Abundant Scenario Plan

```
0.0: (check_levels c5 robot1)
1.0: (move robot1 start1 c1)
2.0: (move robot1 c1 c4)
3.0: (move robot1 c4 c5)
4.0: (irrigate robot1 c5)
5.0: (irrigate robot1 c5)
6.0: (check_levels c1 robot1)
7.0: (move robot1 c5 c2)
8.0: (move robot1 c2 c1)
9.0: (irrigate robot1 c1)
10.0: (irrigate robot1 c1)
11.0: (irrigate robot1 c1)
12.0: (irrigate robot1 c1)
13.0: (check_levels c5 robot1)
14.0: (move robot1 c1 c2)
15.0: (move robot1 c2 c5)
16.0: (irrigate robot1 c5)
17.0: (irrigate robot1 c5)
18.0: (check_levels c3 robot1)
19.0: (move robot1 c5 c2)
20.0: (move robot1 c2 c3)
21.0: (irrigate robot1 c3)
22.0: (check_levels c5 robot1)
23.0: (move robot1 c3 c2)
24.0: (move robot1 c2 c5)
25.0: (irrigate robot1 c5)
```

This plan successfully completes in 25 steps with `num_sacrificed = 0`, consuming exactly 100 units of water (10 iterations $\times$ 10 units). Notably, the robot does not fully irrigate a single crop to completion before shifting position. Instead, it interleaves partial irrigations across `c5` and `c1` in an oscillatory, back-and-forth traversal pattern. This is a direct consequence of `check_levels` being re-evaluated after individual irrigation steps due to the `(needs_check)` predicate. After two initial irrigations, `c5` reaches a moisture level of 25; because `c1` (17) is now lower, `check_levels` triggers on `c1`, making it the new active priority. This reactive behavior continues until all targeted crops reach ≥ 50. Since the water supply is abundant, the motion overhead carries no objective penalty, and the planner accepts this path as valid.

3.2 Limited Scenario Plan

```
0.0: (check_levels c5 robot1)
1.0: (move robot1 start1 c1)
2.0: (move robot1 c1 c4)
3.0: (move robot1 c4 c5)
4.0: (irrigate robot1 c5)
5.0: (irrigate robot1 c5)
6.0: (irrigate robot1 c5)
7.0: (check_levels c1 robot1)
8.0: (move robot1 c5 c2)
9.0: (move robot1 c2 c1)
10.0: (irrigate robot1 c1)
11.0: (irrigate robot1 c1)
12.0: (check_levels c5 robot1)
13.0: (move robot1 c1 c2)
14.0: (move robot1 c2 c5)
15.0: (irrigate robot1 c5)
16.0: (irrigate robot1 c5)
17.0: (check_levels c1 robot1)
18.0: (move robot1 c5 c2)
19.0: (move robot1 c2 c1)
20.0: (irrigate robot1 c1)
21.0: (sacrifice c3 robot1)
22.0: (sacrifice c1 robot1)
```

This plan terminates with `num_sacrificed = 2`. The robot first waters `c5` up to a safe level ($5 \times 10 = 50$ units spent; raising `c5` from $5 \rightarrow 55$), then moves to `c1` to apply 3 steps ($3 \times 10 = 30$ units spent; raising `c1` from $17 \rightarrow 47$), which completely exhausts the 80-unit pool. With water depleted, `c1` cannot reach the safety baseline and is sacrificed. The robot then moves to `c3` (moisture remaining at 42) and sacrifices it as well.

The structural cause is the strict constraint of `check_levels`, which prevents targeting `c1` until `c5`'s moisture is at least equal to `c1`'s value (17). Coupled with `c5`'s massive deficit, 50 of the available 80 units are spent entirely on `c5`, leaving only 30 units for the rest of the field. This demonstrates how strict priority constraints combined with limited resources force the planner to sacrifice multiple crops, even when their combined deficit is lower than that of a single highly-depleted crop.

4 PDDL+ – Continuous Domain Dynamics

PDDL+ introduces continuous temporal features to model environmental mechanics more accurately. This includes passive continuous phenomena and changing instantaneous actions into duration-based actions. Continuous dynamics include soil moisture evaporation and sudden drought hazards, modeled as a process and an event respectively:

```
(:process evaporation
    :parameters (?c - crop)
    :precondition (and (>= (moisture_level ?c) 10) (not (unusable ?c)))
    :effect (and
        (decrease (moisture_level ?c) (* 1 #t))
    )
)
```

Listing 5: PDDL+ Evaporation Process

```
(:event drought
    :parameters (?c - crop)
```

```
    :precondition (and
        (<= (moisture_level ?c) 10)
        (not (unusable ?c))
    )
    :effect (and
        (assign (moisture_level ?c) 0)
        (unusable ?c)
        (increase (num_drought_events) 1)
    )
)
```

Listing 6: PDDL+ Drought Event

The `evaporation` process continuously drains a crop's moisture at a rate of 1 unit per second. If a crop's moisture falls to 10 or below, it triggers the `drought` event, which flags the crop as `unusable` and increments a penalty metric.

To reflect execution durations, actions like movement and irrigation are modeled as continuous processes bounded by discrete start and stop triggers. The robot's condition is tracked via an `(idle ?r)` predicate. Target selection is governed by the following logic:

```
(:action choose_target
    :parameters (?r - robot ?c - crop)
    :precondition (and
        (idle ?r)
        (forall (?other - crop) (<= (moisture_level ?c) (moisture_level
    ?other)))
        (not (unusable ?c))
        (> (moisture_level ?c) 10))
    :effect (and (not (idle ?r)) (targeted ?c))
)
```

Listing 7: PDDL+ Target Selection Action

The robot targets the usable, non-drought crop with the globally lowest moisture level and clears its `idle` state.

Irrigation is managed by two discrete actions that bracket a continuous watering process (`irrigating_crop` which yields a net moisture increase of +9/sec due to matching evaporation):

- `start_irrigation`: Sets the `(irrigating ?r ?c)` flag once the robot arrives at the targeted crop, initiating continuous water delivery.
- `stop_irrigation`: Clears the continuous irrigation flags and returns the robot to an `idle` state once the crop's moisture strictly exceeds 50.

```
(:action start_irrigation
    :parameters (?r - robot ?c - crop)
    :precondition (and (at ?r ?c) (not (irrigating ?r ?c)) (targeted ?c
   ) (> (water_supply ?r) 0))
    :effect (and (irrigating ?r ?c))
)

(:action stop_irrigation
    :parameters (?r - robot ?c - crop)
    :precondition (and (at ?r ?c) (irrigating ?r ?c) (> (moisture_level
    ?c) 50))
    :effect (and (not (irrigating ?r ?c)) (idle ?r) (not (targeted ?c))
   )
```

```
)
```

Listing 8: PDDL+ Irrigation Control Actions

Using discrete actions to regulate the irrigation process ensures that the robot reacts immediately once a crop is safe, allowing it to move to other endangered areas without delay.

Robot transit is managed by a continuous `moving` process initiated by a discrete `start_move` action. When transit begins, the robot's old position is cleared, a `next_location` is assigned, and a `move_progress` variable is set to 0. The `moving` process increments `move_progress` by 1 unit per second. After 1 second has elapsed, an `arrive` event triggers, updating the robot's position to the destination and clearing the transit flags. This step-by-step approach simplifies multi-hop navigation across connected locations and avoids instantaneous movement, providing a more accurate simulation of transit times.

5 PDDL+ – Problem Definition and Evaluation Results

A problem instance (`Irrigation_problem_plus.pddl`) was formulated to validate the continuous domain model. The variables `num_drought_events` and `move_progress` are initialized to 0, and the initial water supply is set to 200 units. To compensate for the immediate toll of continuous evaporation during transit, initial crop moisture profiles were adjusted compared to the discrete scenario: `c1:27, c2:63, c3:42, c4:88, c5:25, c6:71`.

The optimization objective uses a composite metric that minimizes total execution time while heavily penalizing drought occurrences:

```
(:metric minimize (+ (total-time) (* 1000 (num_drought_events))))
```

Listing 9: PDDL+ Optimization Metric

The goal formulation requires that all crops are kept safe and that the robot eventually shuts down its systems to return to a baseline state:

```
(:goal (and
    (forall (?c - crop) (or (> (moisture_level ?c) 50) (unusable ?c)))
    (idle robot)
))
```

The complete execution plan generated by the ENHSP solver under the `aibr` heuristic is detailed below:

```
Found Plan:
0: (choose_target robot c5)
0: (start_move robot start1 c1)
0: -----waiting---- [1.0]
1.0: (start_move robot c1 c4)
1.0: -----waiting---- [2.0]
2.0: (start_move robot c4 c5)
2.0: -----waiting---- [4.0]
4.0: (start_irrigation robot c5)
4.0: -----waiting---- [8.0]
8.0: (stop_irrigation robot c5)
8.0: (choose_target robot c1)
8.0: (start_move robot c5 c4)
8.0: -----waiting---- [9.0]
9.0: (start_move robot c4 c1)
```

```
9.0: -----waiting---- [10.0]
10.0: (start_irrigation robot c1)
10.0: -----waiting---- [16.0]
16.0: (stop_irrigation robot c1)
16.0: (choose_target robot c3)
16.0: (start_move robot c1 c2)
16.0: -----waiting---- [17.0]
17.0: (start_move robot c2 c3)
17.0: -----waiting---- [18.0]
18.0: (start_irrigation robot c3)
18.0: -----waiting---- [22.0]
22.0: (stop_irrigation robot c3)
22.0: (choose_target robot c2)
22.0: (start_move robot c3 c2)
22.0: -----waiting---- [23.0]
23.0: (start_irrigation robot c2)
23.0: -----waiting---- [25.0]
25.0: (stop_irrigation robot c2)
25.0: (choose_target robot c5)
25.0: (start_move robot c2 c5)
25.0: -----waiting---- [26.0]
26.0: (start_irrigation robot c5)
26.0: -----waiting---- [28.0]
28.0: (stop_irrigation robot c5)
28.0: (choose_target robot c6)
28.0: (start_move robot c5 c6)
28.0: -----waiting---- [29.0]
29.0: (start_irrigation robot c6)
29.0: -----waiting---- [30.0]
30.0: (stop_irrigation robot c6)
```

The priority logic operates as intended, targeting the most depleted crop (`c5`) first. Continuous dynamics alter plan generation over time. For example, `c2` is selected as a target at $t = 25$ seconds, despite starting well above the safety baseline, because continuous evaporation gradually lowered its moisture level.

At $t = 30.0$ seconds, the last crop (`c6`) reaches a moisture level of 51, satisfying the moisture goal. Due to the `(idle robot)` constraint embedded in the problem's goal, the planner introduces the final `(stop irrigation robot c6)` action at $t = 30.0$ to successfully wrap up the plan and validate the terminal state.

This plan highlights several key trade-offs between resource allocation and overall crop health:

- **Water Efficiency Suboptimality**: To prevent drought events, the planner may increase total water consumption. Continuous evaporation affects all crops simultaneously, requiring the robot to revisit and water crops like `c2` and `c6` that were initially safe.
- **Transit Overhead**: While step-by-step movement provides a reliable navigation strategy, it increases total field evaporation. Because the robot addresses one crop at a time, each transit step allows moisture levels across the entire field to decrease without active replenishment.
- **Resource Constraints**: In more challenging scenarios, limited water availability can lead to unavoidable drought events. Addressing these limitations requires either expanding tank capacities or deploying multi-robot systems.